PROJECT 2 REPORT

Coin-picking robot

The University of British Columbia

Electrical and Computer Engineering department

Electrical Engineering Design Studio 1

ELEC 291

Instructor: Dr. Jesus Calvino-Fraga

Section: 201

Group: B08

Student #	Student Name	Points (%)	Signature
99991564	Jackson Rockford	116	Jackson R
52130176	Allen Pham	97	
93279289	Matthew Yuen	107	Matthew
31929185	Adrian Chua	80	
44955458	Harry Nguyen	100	

Disclaimer:

We, members of group B08, hereby declare that the table above accurately reflects our individual contribution to the project, and that the numbers shown are unanimously agreed upon by all members of this group.

Table of Contents

PROJECT 2 REPORT ***1***

INTRODUCTION **3**

INVESTIGATION: **7**

DESIGN **9**

SOLUTION ASSESSMENT **20**

LIVE LONG LEARNING **22**

CONCLUSION **22**

APPENDIX: **23**

BIBLIOGRAPHY: **25**

INTRODUCTION

Overview:

Throughout previous labs and projects of ELEC 291, and indeed of other courses like ELEC 201 and CPEN 211, we have been exposed to a variety of different electrical components and microcontrollers. Both in terms of theory, as well as their practical usages under different applications. This project aims to bring the bulk of our knowledge of said components together to create a robot that can detect and pick up coin, while adhering to serval design constraints and objective.

Objective:

The goal of this project is to build, program, and test a robot, capable of detecting and pick up coins, both autonomously and manually:

- The project must utilize two microcontrollers (one for the robot and one for the remote). The two microcontrollers must be from **different families**. The list of MCUs included in the **project 2 kit** is as below:
- Both the robot and controller must be battery operated (AA and 9V)
- **Metal detection:** using a 1mH conductor coil and other relevant electrical components to construct a Colpitts Oscillator, which will act as a metal detector. The robot must be able to detect all Canadian coins in circulation.
- **Coin-picking mechanism** consists of two micro servo motors and an electromagnet. The robot must be able to pick up all Canadian coins in circulation and deposit them in a container situated at the front of the robot.
- **The robot must be programmed in C**.

- **Perimeter detection**: in automatic mode, the robot must operate within a 1m x 0.5m perimeter. The perimeter consists of a long rectangular wire carrying an AC signal either from a function generator or a LM-555 timer circuit. The robot must be able to detect and stay within the perimeter at any given point throughout the automatic process.
- **Automatic mode**: the robot must be able to autonomously pick up 20 coins scattered randomly within the perimeter, without having four wheels leaving said perimeter. Upon completion, the robot stopped for further commands.
- **Manual mode**: the controller must be able to control the robot using a remote. The basic manual controls are move forward, backward, left, right and pick up a coin.
- **Remote control:** must include an LCD to display the signal “strength” coming from the coin detector, as well as a speaker that beeps whenever a coin is detected. The pitch of the speaker must correspond to the strength of the signal coming from the metal detector. A joystick is included in the project 2 kit, although any other suitable control methods are also allowed.

Hardware and Software block diagram:

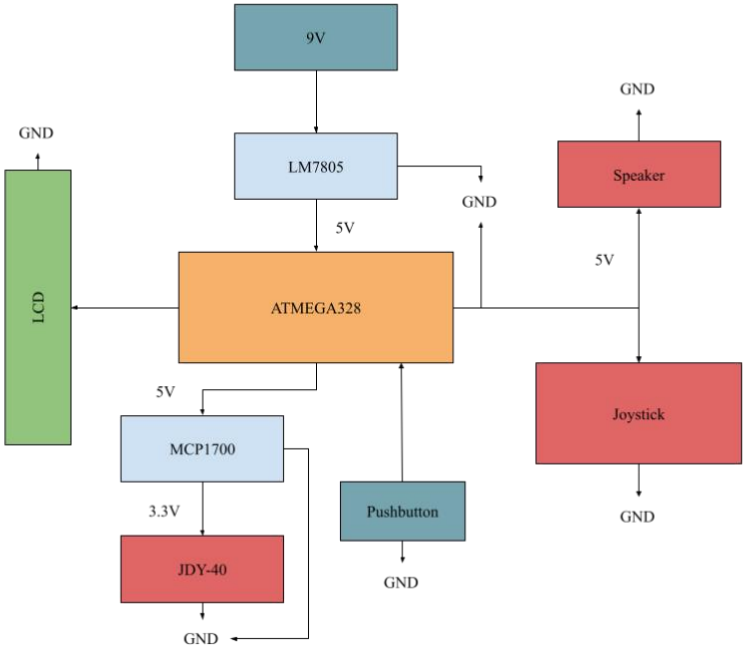

Figure 1 Remote Block Diagram

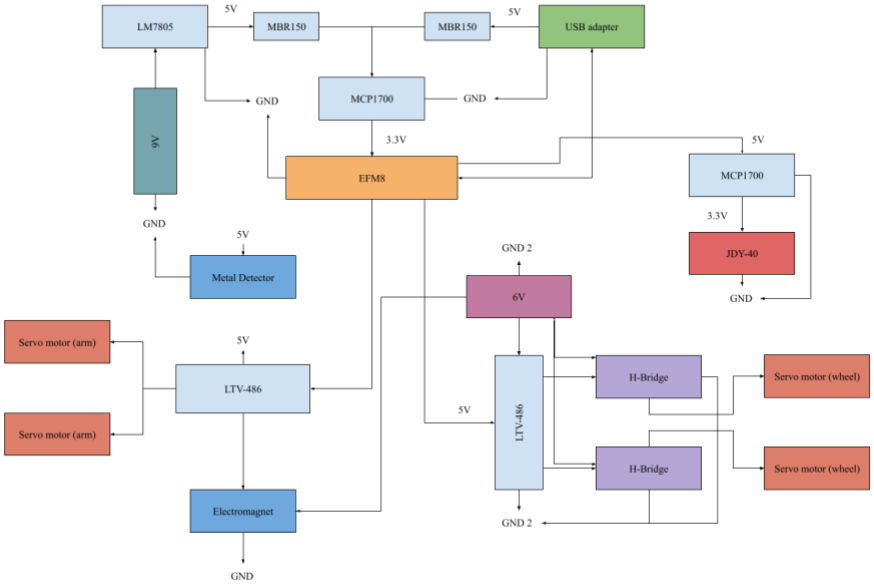

Figure 2 Robot Block Diagram

Figure 3 Master-code (remote) Block Diagram

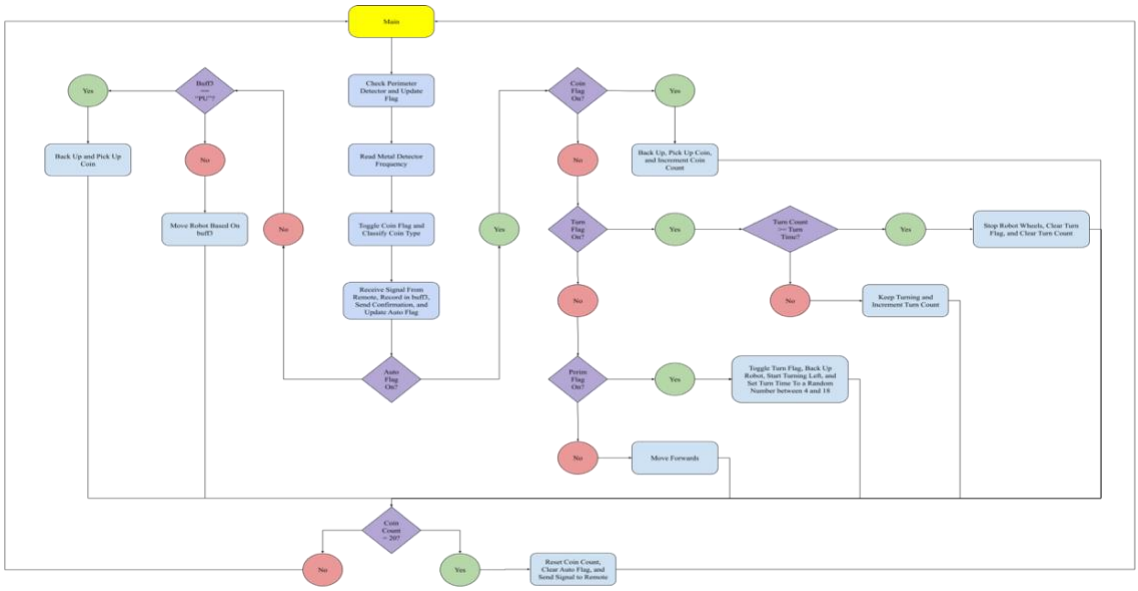

Figure 4: Slave-code (Robot) Block Diagram

INVESTIGATION:

Idea Generation:

Ideas were generated and tackled base on design specifications and objectives listed above. For this project, the keys areas to focus on were:

- Succinct integration of two different micro controller families via the C.
- Discrete MOSFETs to drive motor logic.
- Metal and perimeter detector circuits and logic.
- Radio communication via the JDY-40 radio.
- Remote must have LCD display, speaker and joystick

Once project objectives were identified, our group investigated each aspect as a whole and attempted to come up with a solution. Progress was monitored through regular check-ins and testing to ensure succinct integration at the end.

Investigation Design:

- Our group's first objective was deciding which microcontroller unit to use. At the end, we settled with the ATMEGA328P for the controller, and the EFM8 (included in our project 1 kit) for the robot. This decision stemmed from our extensive experience with both MCUs from previous labs and projects, as well as sufficient documentation to both [(1)(2)].
- For the metal detector circuit, there are multiple designs available on the Project 2 lecture slides [(3)]. Our team built and tested all the designs and eventually settled the Colpitts Oscillator with discrete CMOS inverter. This set up ensured

the most stable frequency output when a coin was detected, as well as being ergonomic enough to be incorporated into the overall robot design.

Data Collection:

Dedicated procedures to test the metal and perimeter detector were carried out to ensure succinct operation.

- By placing the robot at various position and distance relative to the perimeter, the "ambient" perimeter frequency of the perimeter detector was consistently found to be around 56600Hz. We set this as our reference value. The frequency picked up by the metal detector when different types of coins were detected varied. Each coin type had a "signature" range of frequency. We later take the average

Ambient (Hz)	Max (Hz)	Min (Hz)	Toonie (Hz)	Max (Hz)	Min (Hz)	Loonie	Max (Hz)	Min	Quarter	Max (Hz)	Min (Hz)	Dime	Max (Hz)	Min (Hz)	Nickel	Max (Hz)	Min (Hz)
56426	56568	56197	58027	58027	57655	57692	58045	57655	57618	57673	57379	56854	57034	56657	57215	57507	57106
56250			57766			57840			57618			56854			57270		
56338	Avg		57822	Avg		57822	Avg		57434	Avg		56872	Avg		57324	Avg	
56250	56380.64		57840	57816.02		57840	57831.54		57581	57603.46		56854	56843.54		57324	57308.22	
56390			57822	Diff		57729	Diff		57618	Diff		56836	Diff		57324	Diff	
56408			57673	1435.376		58027	1450.897		57618	1222.824		56692	462.8958		57233	927.5791	
56444			57822			57859			57600			56854			57343		
56426			57822			57859			57618			56872			57324		
56426			57822			57710			57618			56800			57489		
56267			57840			57840			57600			56908			57343		
56426			57692			57859			57600			56854			57306		
56426			58027			57859			57655			56854			57343		
56408			57822			57896			57618			56872			57161		
56444			57803			57859			57600			56872			57397		
56408			57784			57859			57618			56854			57270		
56426			57840			57803			57600			56872			57270		
56250			57673			57692			57655			56854			57507		
56214			57840			57840			57618			56836			57324		
56426			57822			57859			57636			56675			57343		
56250			57840			57859			57600			56854			57324		
56320			57822			57692			57600			56836			57142		
56568			57673			57840			57618			56800			57288		

Figure 5:5 Frequency variation across different types of coin

of each coin type's frequency and their difference from the ambient metal detector frequency (i.e. when nothing is in the vicinity of the metal detector) as a base to drive our metal detector logic.

Data Synthesis:

The data found regarding the frequency generated by both the metal and perimeter detector under different conditions served as an important foundation for the robot logic.

The accuracy of data collected were further verified using an oscilloscope and excel arithmetic. Via C, we hardcoded the baseline frequency for both detectors. These baseline values directly control the motor that moves the robot around, as well as the coin-picking mechanism (both the servo “arm” and the electromagnet). Once sufficient data was achieved, it all came down to simple arithmetic and if-else statements in the C-language to drive the robot logic.

Analysis of result:

Through troubleshooting and calibration, we mitigated errors due to hardware limitations and environmental factors with regards to both the metal and perimeter detector. Our analysis confirmed both apparatuses operating within acceptable error margins and frequency data used as baseline values to drive the robot logic resulted in accurate, consistent and predictable robot behavior.

DESIGN

Use of Process

Our team began with a structured engineering approach to ensure each subsystem was reliably tested and validated before moving on to subsequent phases of the project. First, we focused on the hardware: we assembled the robot chassis (wheels, motors, coin picking assembly, electromagnet, inductor coils for metal and perimeter detector and basket), and tested the key components—such as the Colpitts oscillator and simple perimeter detector circuits, both microcontrollers (ATMEGA328 and EFM8), JDY-40 radios—to confirm they functioned correctly. By establishing a stable hardware

baseline, we minimized the risk of falsely attributing errors during software development to hardware issues.

Requirements and Constraints Identification

We gathered requirements by discussing baseline robot operations based on the outlined requirements, carefully examining the lecture slides and reviewing relevant best practices for designing and operating a multi-pronged system such as this project. From these activities, we identified the following needs and constraints:

- **Streamlined controls during manual mode**: the remote (both in terms of hardware and software) must be designed to allow for intuitive and ease of control during manual operations. Steps must be made to always allow for smooth and accurate controls with little interruptions, glitches or delay.
- **Optimized robot logic during automatic operation**: the robot must be able to perform multiple tasks simultaneously at any given point. This emphasizes the need for a clear logic flow that will allow for optimal performance and ease of debugging.
- **Succinct integration between automatic and manual mode**: The operator must be able to switch between automatic and manual mode at any given time. Once in automatic mode, the robot must strictly adhere to the base requirements of this project (i.e. stays withing the perimeter, automatically detects and picks up coin). In addition, after the 20 coins have been picked up during the automatic process, the robot should halt and awaits further instructions from the operator.

- **Relevant Information Display**: The remote should (at the bare minimum) display the signal strength based on which type of coins are detected. The signal strength must also be represented by an audio cue via a speaker.

By systematically identifying and documenting these needs, we ensured our design choices directly addressed user requirements and practical constraints from the outset, helping guide all subsequent design and development efforts.

Solution Generation

Our team generated solution systematically and took into consideration various methods to address the project's core requirements. We started by investigating different hardware and software configurations to deal with the robot's various operational requirements.

1. Controls:
    - We opted for a simple, 2-axis control via the provided joystick when it comes to the movement of the robot.
    - The robot arm can be activated by pressing down on the joystick
    - Toggling between manual and automatic mode can be done via a pushbutton
2. Radio communication between robot and remote:
    - The ATMEGA328 microcontroller had several built-in ADC input pins, which, when connected to the joystick, can translate its movement into ADC signal and then to voltage. We then programmed the movement of

the robot by figuring out the joystick's "dead zone" and finetuning it to fit with the H-bridge logic, which is then used to move the robot.

3. Software Implementation:
    - Our team decided to drive the robot's logic by imbedding different parameters as presets. From the frequency picked up by the perimeter and metal detector, to the text command signals sent and received from the robot to the remote, we were able to systematically work out the robot's core operation via simple while loops and if-else statements. This greatly enhanced the efficiency of our workflow, allowed for ease of debugging and optimized the robot's performance and accuracy.

Solution Evaluation

After potential solutions were generated, we evaluated each of them based on accuracy, ease of implementation and how well each idea interacts with one and another in the context of this project. Our evaluation criteria included:

- Hardware reliability: assuring an ergonomic, neat and efficient circuit design that minimizes errors and allows for easier modification and troubleshooting, as well as succinct integration with software.
- Software efficiency: ensuring an optimal and logical code flow that once again highlights the need for ease of troubleshooting and modification.
- Efficient and predictable robot operation.

Key Findings:

Although a simple control scheme and clear logic-based code fast-tracked the development of our design, we encountered multiple hiccups regarding to the communication between the robot and remote, which caused unpredictable robot movements, delays and crashes. We eventually narrowed down the cause of the issue: the radio communication received by the robot from the remote were somehow being offset. To be more specific:

- For example, during manual operation, when the robot is commanded to move forward, the operator moves the joystick forward, which will send a series of signal to the robot, the continuous signal looks something like this: “!FN!FN!….” The software of the robot was supposed to only grab “FN”, which was the valid command for “forward neutral”, but sometimes, it picked up “!F”, which was a false command based on our existing logic. This happened consistently for other commands as well, causing the robot to behave unpredictably. We were able to fix this issue by hardcoding the invalid commands (i.e. “!F”, etc.) as valid instructions. This fixed the issue, while maintaining the overall code efficiency and logic flow:

```
if (strcmp(command, "FN") == 0 || strcmp(command,"!F") == 0)
{
```

Figure 6: Conditional statement that takes care of the offset issue

Detailed Design

The design of this project integrates both hardware and software to ensure succinct user interaction, data processing and process automation.

Hardware design:

The robot system was composed of multiple functional components: the robot chassis motor drive, microcontroller, metal and perimeter detectors, wireless radio communication and coin-picking mechanism. Each subsystem was designed by applying principles of embedded systems, A/D design circuit design and power management.

- Power regulation:
    - Design approach: The robot uses 4x AA batteries (6V) to power the motors and optocouplers, and a 9V battery regulated to 5V by an LM7805 and an MCP1700 for 3.3 V.
    - LM7805: provides 5V for the EFM8 and sensors.
    - MCP1700: provides 3.3V for JDY-40 and microcontroller
- Microcontroller system:
    - Robot MCU – EFM8:
        - 4 PWM digital outputs for motor control, UART + SET pins for JDY-40 communication, 1 digital input for metal detector, and 4 analog inputs for perimeter detectors
    - Remote MCU – ATMEGA328:
        - 6 digital outputs for LCD, 2 analog inputs for joystick, 1 digital output for speaker, 3 UART pins for JDY-40
- Motor driver: H-bridge and optocouplers

    - 2 discrete H-bridges, each consists of N-MOS and P-MOS transistors driven by optocouplers (LTV-847), control 2 DC motors by isolation motor power circuitry from the logic-level control.
- Metal detector:
    - A Colpitts Oscillator with discrete CMOS inverter built from discrete components.
    - Powered at 5V
    - The frequency shift when metal is in the vicinity is detected by the MCU
- Perimeter detectors:
    - A peak detector circuit, whose signal is amplified using an Op-Amp.
    - Two were put perpendicular to each other at the front of the robot chassis for all-direction sensitivity
    - Output is fed to analog pins of the EFM8 for perimeter detection logic
- Radio communication: JDY-40 radio pair
    - Uses RXD, TXD and SET pins for serial communication between robot and remote.
    - Powered by 3.3 V and communicates at 9600 baud
- Joystick, speaker and LCD (remote):
    - Joystick: 2-axis analog potentiometer-type joystick connected to 2 analog inputs
    - LCD: standard 14-bit display driven by MCU digital outputs
    - Speaker: generates PWM audio cues via a single digital output
- Coin-picking mechanism:

- Utilizes an electromagnet, servo motors and chassis-integrated basket to pick up metal coins.

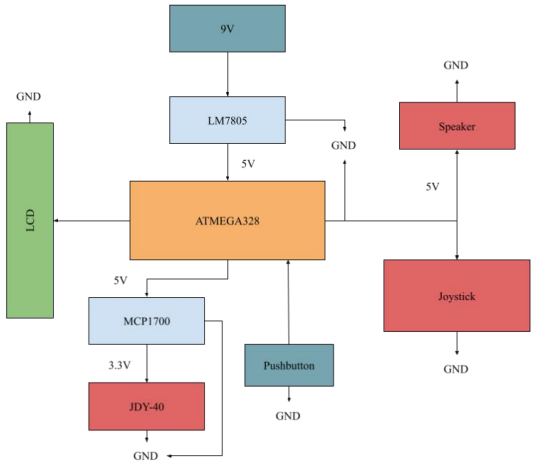

Figure 6 Remote Block Diagram

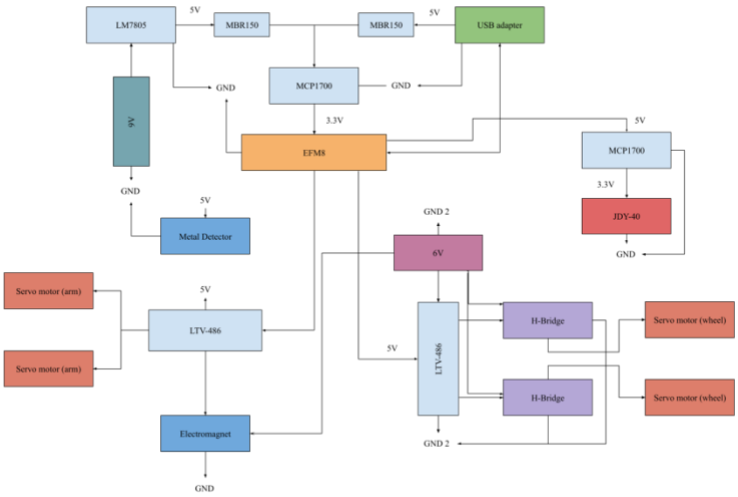

Figure 7 Robot Block Diagram

Software design:

1. Slave code:

- Servos control: Implemented in Timer5_ISR, creating PWM signals with a 20ms period
- Sensor integration:
    - Metal detector: coins are detected via frequency shift
    - Perimeter detectors: voltage threshold logic triggers a perimeter flag
- Movement control:
    - Motor logic implemented via move_robot() function; encoded **movement commands** include:
        - “FN”**:** Forward (2-wheels moving forward), “BN”: Backwards (2-wheels moving backward), “HL” / “HR”: Hard-left, Hard-right (1-wheel moving forward, other backward), “FL” / “FR”: Gradual-left, Gradual-right (Only one wheel moving forward), BL” / “BR”: Back-left, Back-right (Only one wheel moving backward) and “ST”: Stop
- Coin-picking mechanism:
    - Servos_magnet() executes a defined range of motion sequence to pick up coins upon detection
- Wireless communication:
    - Slave-to-master transmits coin data types (“!DO” - Dollar, “!TW” – 2 Dollars, “!NI” – Nickel, etc.)
    - “TO” – toggles between manual and automatic modes
    - “PU” - triggers the coin pick-up sequence

    - Unknown commands are handled (see ***Key findings***, above)

- Automatic mode logic:
    - Moves forward by default
    - If perimeter is detected, triggers *perimeter_flag*, then executes a range of motion moving backward then turning right (the degree of turn is randomized)
    - If metal is detected, uses frequency difference baseline to:
        - Detect presence of coin (triggers *coinflag*); Classify coin types (*coin_tier*); Broadcast coin classification to master (remote); Activate *servos_magnet()* to collect coin
    - Coin collection is hardcoded to 20. After 20 pick-ups, the robot stops. Note that the robot has no means to detect a successful pick-up. It counts every activation of the coin-picking assembly as an attempt.
- Manual mode logic:
    - In manual mode, commands are received wirelessly and processed directly
    - “PU” initiates the pick-up sequence
    - Directional and stop commands are passed to *move_robot()*

2. Master Code:

- Core features:
    - Joystick-based control: reads analog joystick input to send movement commands.

- Mode toggle: a pushbutton to toggle between automatic and manual mode by sending “!TO” and waits for “!TD” / “!TF confirmation
- Pressing down on the joystick to send “!PU” to activate the coin collection sequence

- JDY-40 communication:
  - Sends commands with a “!” header (e.g., “!FN”, “!ST”, “!PU”, etc.)
  - Receives coin types identifiers from the robot (“DI”, “QU”, “NI”, “TW”, “DO”) and toggles “TF” from the robot
- Speaker feedback:
  - Uses Timer0 interrupts to generates tones with different frequencies based on coin type:
- LCD feedback:
  - Two-line LCD display:
    - Current mode (“Toggle On/Off”) and coin strength
    - Display and updates joystick command (e.g. Forward, etc.)

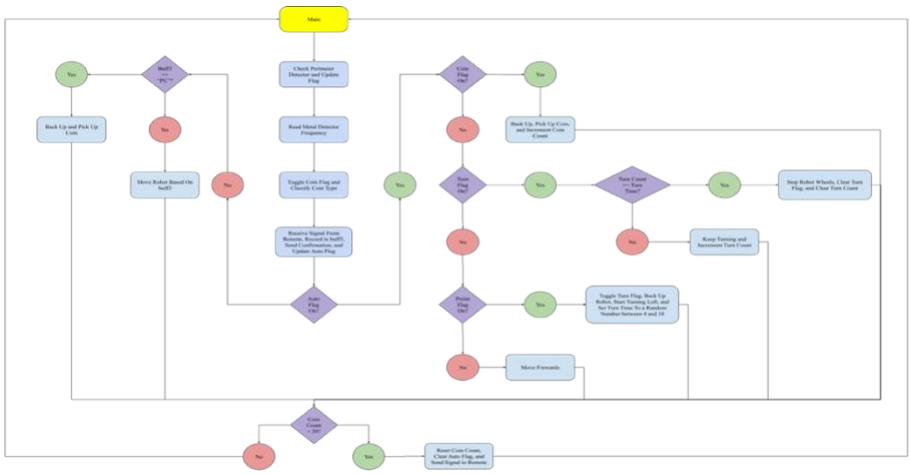

Figure 9: Slave-code (Robot) Block Diagram

Figure 10: Master-code (remote) Block Diagram

SAFETY AND PROFESSIONALISM:

All team members strictly adhered to lab safety rules and upheld professionalism and work ethic:

- Appropriate dress code
- Safety gears whenever required
- Consulted datasheets when it comes to sensitive electrical components
- Work was divided equally and equitably among all members

SOLUTION ASSESSMENT

- The robot was tested over 20 independent trials across various coin placement within the perimeter

- Consistency of successful number of coins pick up per run remained at **18-19 out of 20 coins (± 1)**
- **List of performed tests and observations:**

Test	Expected outcome	Observed outcome	Pass/Fail
Coin detection (static and on the move)	Coin correctly identified and picked up	Coin detection is 100% functional, but hardcoded movement after pickup only allowed for 80-90% successful pick-up rate.	Pass (Partial)
Boundary detection	Robot reroutes away from the perimeter	Correct response 100% of the time	Pass
Remote toggle and command response	Mode toggles + joystick driving	Works reliability with LCD feedback with minor inconsistency with auto/manual mode toggle	Pass (Partial)
Coin classification	Correct sound + LCD	All coin types correctly identified	Pass

LIVE LONG LEARNING

- This project further reinforces various concepts that were introduced in ELEC 201, CPEN 211 and ELEC 202.
- This project gave us foundational safety and programming experience for working with automated devices.
- As electrical engineering students, through this project, we gained valuable experiences in hardware design, programing skills and circuit analysis, which serves as a solid foundation to future labs, projects and career prospect.

CONCLUSION

- This project succeeded in creating a functional coin-picking robot that met the design objectives (see above).
- In addition to meeting the core design objectives, this group also implemented an extra feature that greatly enhanced the overall functionality of the robot: IR sensors that completely replaces the perimeter detectors. These acted as a failsafe in case the function generator was invertedly turned off or sudden malfunctions with regards to the perimeter inductor coils.
- Overall, the project took our group close to 300 hours, over the course of 3.5 weeks.

APPENDIX:

Bill of Materials (BOM):

ROBOT		
	Components	Quantity
1.	EFM8LB12	1.0
2.	LTV846	2.0
3.	LM358P	1.0
4.	LM7805	2.0
5.	MCP1700	1.0
6.	FQU8P10	5.0
7.	FQU13N06LTU	6.0
8.	1N4148	1.0
9.	M8275-ND	3.0
10.	QRD114	2.0
11.	MBR150	2.0
12.	Capacitor 10µF	2.0
13.	Capacitor 1nF	1.0
14.	Capacitor 10nF	1.0
15.	Capacitor 100nF	2.0
16.	Resistor 330Ω	2.0

17.	Resistor 1KΩ	7.0
18.	Resistor 4.7KΩ	1.0
19.	Resistor 100KΩ	4.0
20.	Resistor 330KΩ	2.0
REMOTE CONTROLLER		
1.	ATMega328P	1.0
2.	CEM-1203(42)	1.0
3.	FT230XSR	1.0
4.	FQU13N06LTU	1.0
5.	JDY-40	1.0
6.	MCP1700	1.0
7.	1N4148	1.0
8.	QYF-860	1.0
9.	Resistor 2KΩ	1.0
10.	Resistor 1KΩ	5.0
11.	Push buttons	3.0
12.	Capacitor 0.1µF	4.0
13.	Capacitor 100µF	1.0

Source code:

See submitted Zip file on canvas

BIBLIOGRAPHY:

- A. S. Sedra and K. C. Smith, *Microelectronic Circuits*, 4th ed. New York: Oxford University Press, 1998.
- Silicon Labs, "EFM8LB1-SLSTK2030A User's Guide," 2023. [Online]. Available: https://www.silabs.com/documents/public/user-guides/ug126-efm8lb1-slstk2030a.pdf
- Silicon Labs, "EFM8 Laser Bee Family EFM8LB1 Reference Manual," 2021. [Online]. Available: https://www.silabs.com/documents/public/reference-manuals/efm8lb1-rm.pdf
- ABRA Electronics, "JDY-40 Wireless Serial Port Module," 2022. [Online]. Available: https://abra-electronics.com/index.php?dispatch=attachments.getfile&attachment_id=669
- Microchip Technology Inc., "ATmega48A/PA/88A/PA/168A/PA/328/P megaAVR Data Sheet," 2019. [Online]. Available: https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ProductDocuments/DataSheets/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf
- M. H. Department of Electrical and Computer Engineering, "Programming the Microchip ATmega328P in C," University of Southern California, 2023. [Online]. Available: https://ece-classes.usc.edu/ee459/documents/AVR-gcc.pdf

- J. Carlson, "Getting Started With the ATMega328P," *Instructables*, 2015. [Online]. Available: https://www.instructables.com/Getting-Started-With-the-ATMega328P/
- All About Circuits, "Implementing I2C with an EFM8 Microcontroller," 2015. [Online]. Available: https://www.allaboutcircuits.com/technical-articles/implementing-i2c-with-an-efm8-microcontroller-part-1/
- Longan Labs, "Serial RF Module," 2020. [Online]. Available: https://docs.longan-labs.cc/1040010/
- Kompulsa, "Getting Started With The Silicon Labs EFM8: GPIO Basics," 2023. [Online]. Available: https://www.kompulsa.com/getting-started-silicon-labs-efm8-basics/
- Vyrian, "ATMega328P Microcontroller Pinout, Features & Datasheet," 2024. [Online]. Available: https://www.vyrian.com/blog/atmega328p-microcontroller-features-pinout-configuration-datasheets/

REFERENCES

[1] J. Calvino-Fraga. ELEC 291. Class Lecture, Topic: “EFM8 board, FSM, NVMEM and tips.” Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, BC, Mar. 14, 2025.

[2] J. Calvino-Fraga. ELEC 291. Document, Topic: “The AVR controller system.” Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, BC, 2017

[3] J. Calvino-Fraga. ELEC 291. Class Lecture, Topic: “Project 2 – Coin Picking Robot.” Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, BC, Feb 7th, 2025.